

Robotic Arm ROS Control

1. Function Description

After the program starts, you can use ROS to control the robotic arm's pose and individual servo values. This uses the ROS2 topic command-line tool to change the robotic arm's pose and individual servo values.

2. Startup and Operation

2.1 Startup Command

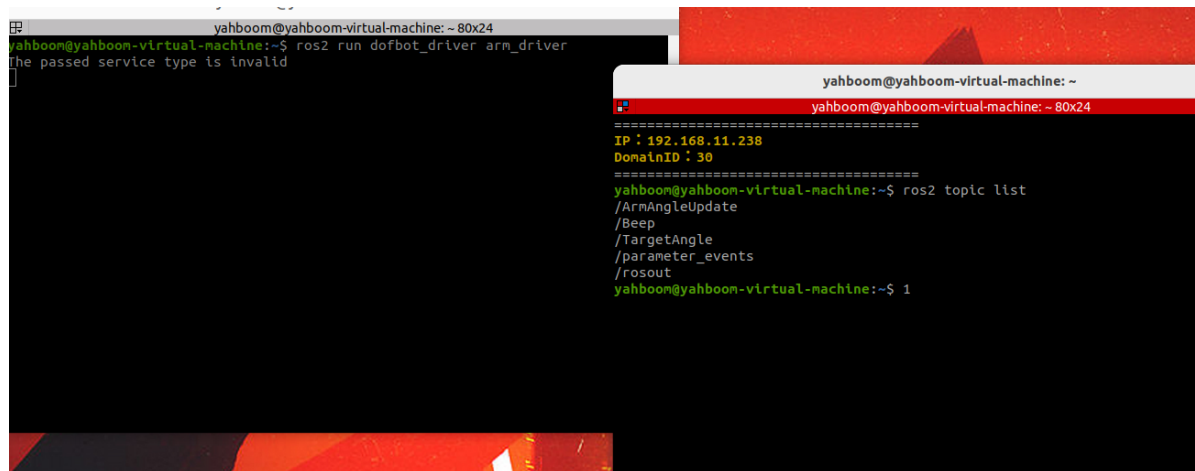
After opening the terminal, enter the following command:

```
# Start the underlying driver
ros2 run dofbot_driver arm_driver
```

2.2 Operation Steps

Start the terminal and enter the following command:

```
ros2 topic list
```



As shown in the image above, the displayed /TargetAngle is the topic for controlling the robotic arm. We publish messages to this topic using the rostopic tool. First, let's check the topic's related information. Enter in the terminal:

```
ros2 topic info /TargetAngle
```

```
yahboom@yahboom-virtual-machine:~$ ros2 topic info /TargetAngle
Type: dofbot_interface/msg/ArmJoint
Publisher count: 0
Subscription count: 1
yahboom@yahboom-virtual-machine:~$
```

As shown in the image above, the data type name of this topic is dofbot_interface/ArmJoint. This is our custom message data interface. You can view the specific content of this data type with the following command. Enter in the terminal:

```
ros2 interface show dofbot_interface/msg/ArmJoint
```

```
yahboom@yahboom-virtual-machine:~$ ros2 interface show dofbot_interface/msg/ArmJoint
oint
int32 id
int32 run_time
float32 angle
float32[] joints
yahboom@yahboom-virtual-machine:~$
```

As shown in the image above, the specific content of the dofbot_pro_interface/ArmJoint type data is displayed. The attributes of each variable are as follows:

- id: corresponds to the servo's id. When controlling a single servo, you need to specify this value. The value range is [1-6];
- run_time: servo running time. Within this time, the robotic arm needs to reach the specified pose or a single servo needs to reach the specified angle. Unit is ms
- angle: control angle. When controlling a single servo, you need to set the angle value after operation. The value range is [0,180]
- joints: an array of length 6. When controlling the values of 6 servos, write the values of the 6 servos into this data. The array index corresponds to each servo's id-1. The control range of each servo angle is [0,180]

We can test this with the following command:

To control a single servo, taking the control of servo #1 as an example, enter in the terminal:

```
ros2 topic pub /TargetAngle dofbot_interface/ArmJoint "id: 1
run_time: 1000
angle: 120.0
joints: []"
```

It is recommended to use Tab on the single quote keyboard to complete the topic name and the content of the following data, then modify the corresponding values. After input is complete, press Enter to send the topic data. Servo #1 of the robotic arm will rotate to the 120° state.

To control multiple servos, taking the robotic arm presenting an upward extended pose as an example, enter in the terminal:

```
ros2 topic pub /TargetAngle dofbot_interface/ArmJoint "id: 1
run_time: 1000
angle: 120.0
joints: [90,90,90,90,90,90]"
```

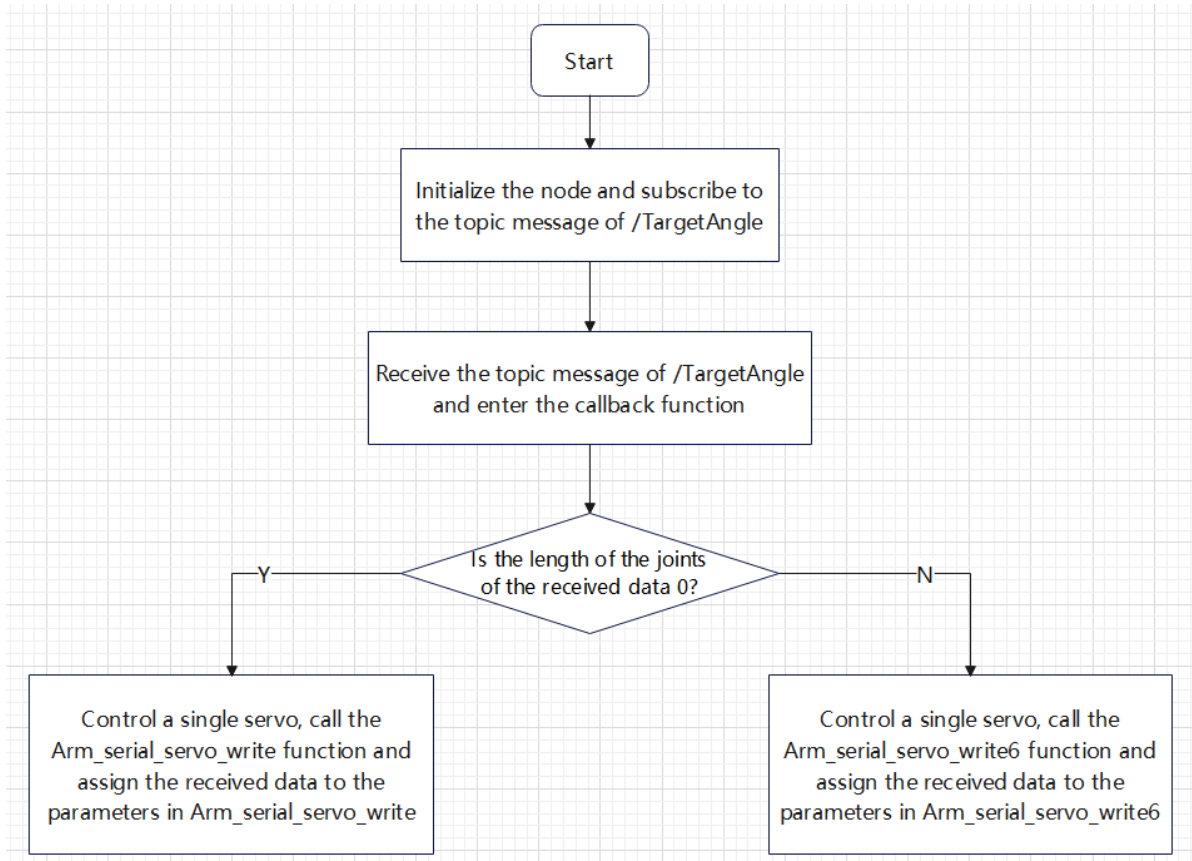
Similarly, we recommend using Tab on the single quote keyboard to complete the topic name and the content of the following data, then modify the corresponding values. After input is complete, press Enter to send the topic data. The robotic arm will present an upward extended pose.

To control the buzzer:

```
# Turn the buzzer on
ros2 topic pub /Beep std_msgs/msg/Bool "data: True" --once

# Turn the buzzer off
ros2 topic pub /Beep std_msgs/msg/Bool "data: False" --once
```

3. Program Flowchart



4. Core Code Analysis

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_driver/dofbot_driver/arm_driver.py
```

Import library files:

```
import rclpy
from rclpy.node import Node
# Robotic arm control library
from Arm_Lib import Arm_Device
from dofbot_interface.msg import ArmJoint
from std_msgs.msg import Bool
import time
from sensor_msgs.msg import JointState
import numpy as np
from math import pi
import os
```

Define a subscriber to subscribe to the TargetAngle topic:

```
self.sub_Arm = self.create_subscription(ArmJoint, "TargetAngle",
self.arm_callback, queue_size=1000)
```

Write a callback function to process the subscribed messages and send data to the underlying control:

```
def arm_callback(self, msg):
    arm_joint = ArmJoint()

    if len(msg.joints) != 0:
        self.get_logger().info(f"Received joints: {msg.joints}")
        arm_joint.joints = self.cur_joints
        for _ in range(2):
            self.Arm.Arm_serial_servo_write6(
                msg.joints[0], msg.joints[1], msg.joints[2],
                msg.joints[3], msg.joints[4], msg.joints[5],
                time=msg.run_time
            )
            self.cur_joints = list(msg.joints)
            self.arm_pub_update.publish(arm_joint)
    else:
        self.get_logger().info(f"Moving joint {msg.id} to {msg.angle}")
        arm_joint.id = msg.id
        arm_joint.angle = msg.angle
        for _ in range(2):
            self.Arm.Arm_serial_servo_write(msg.id, msg.angle, msg.run_time)
            self.cur_joints[msg.id - 1] = msg.angle
            self.arm_pub_update.publish(arm_joint)

    self.joints_states_update()
```

The code checks the length of the received joints, which is `len(msg.joints)` in the code. It determines whether to control 6 servos or a single servo based on the length of joints. If it is not 0, it means **controlling 6 servos**. When publishing data to the underlying layer, we need to **use the `Arm_serial_servo_write6` function to send data**; otherwise, if it is 0, it means **controlling a single servo**, and we need to **use the `Arm_serial_servo_write` function to send data**.

The **`Arm_serial_servo_write6`** function is provided by the `Arm_Lib` library. `Arm_Lib` is our custom library for controlling the underlying driver. `Arm_serial_servo_write6` has 7 parameters, which are the values of the 6 servos and the running time. After receiving the topic data, we just need to input the 6 servo values (joints) and the `run_time` value correspondingly.

The **`Arm_serial_servo_write`** function is also provided by the `Arm_Lib` library. It requires 3 input parameters: the id of the servo to control, the angle to control the servo to, and the running time. After receiving the topic data, we just need to input the id value, the control angle value (angle), and the `run_time` value correspondingly.